

|                                                                                                                                           |                                                                                                                       |                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>Nombre (s) :</b><br>Darío Alberto Sánchez Perzabal<br>Santiago Patricio Gómez Ochoa<br>Ximena Ortiz Gómez<br>María José Croche Álvarez |                                                                                                                       | <b>Matrícula (s):</b><br>A01738266<br>A01735171<br>A01735100<br>A01734561 |
| <b>Nombre del curso:</b><br>Fundamentación de Robótica                                                                                    | <b>Nombre del profesor (es):</b><br>Juan Manuel Ahuactzin Larios<br>Rigoberto Cerino Jiménez<br>Alfredo García Suárez |                                                                           |
| <b>Módulo:</b>                                                                                                                            | <b>Actividad:</b> Reporte Reto semanal 1 Manchester Robotics                                                          |                                                                           |
| <b>Fecha:</b> 18 de febrero de 2026                                                                                                       |                                                                                                                       |                                                                           |

## Resumen

En este primer reto semanal se desarrolló el procesamiento de una señal senoidal, al igual que su visualización, con ayuda de *rqt\_plot*. Para ello se utilizaron dos nodos los cuales se comunican por medio de tópicos. Igualmente se implementó un archivo launch, el cual automatiza la ejecución. Todo esto se elaboró en ROS 2 que a su vez trabaja con orientado a objetos y con una forma publicación- suscripción. Una vez desarrollado el código para la señal tanto original como la desfasada y con cambio de amplitud, se elaboró el archivo launch, con el cual se ejecutó desde la terminal y con ayuda de *rqt\_plot* se observaron las dos señales en la misma ventana, donde se pueden también analizar los cambios que sufre la segunda señal.

## Objetivos

El objetivo principal de este reto semanal consistió en integrar y desarrollar en ROS2 un sistema de procesamiento y visualización de una señal senoidal utilizando dos nodos llamados “signal\_generator” y “process” los cuales debían comunicarse por medio de tópicos y realizar un archivo launch para automatizar la ejecución del programa y de *rqt\_plot*.

Explicando más a fondo lo anterior, el reto de la semana 1 tiene como objetivo implementar un nodo llamado “signal\_generator” que publique la señal senoidal en el tópico “/signal” y que también publique el tiempo transcurrido utilizando mensajes de tipo Float32. De igual manera, se debe implementar un nodo llamado “process” que se suscriba al tópico “/signal” y al tópico “/time”. Además, el nodo “process” debe transformar la señal senoidal aplicando un desfase, una reducción de amplitud y un desplazamiento vertical. Finalmente este mismo nodo debe ser el que publique el resultado del tópico “/proc\_signal”. Otro de los objetivos del reto es visualizar simultáneamente “/signal/data” y “/proc/data” en rqt\_plot, con el fin de poder comparar la onda senoidal original y la onda procesada. Finalmente, el reto también consiste en generar un archivo launch que ejecute ambos nodos y rqt\_plot con los tópicos requeridos.

## **Introducción**

ROS o también conocido como Sistema Operativo Robótico (ROS por sus siglas en inglés) es un framework open source que proporciona herramientas y bibliotecas de software para facilitar la creación de aplicaciones robóticas. ROS es un sistema operativo diseñado para funcionar con varias plataformas y lenguajes de programación, lo que lo convierte en una herramienta flexible. De acuerdo con Open Robotics (2021), ROS provee servicios de abstracción de software, control de dispositivos de bajo nivel, pasaje de mensaje entre procesos y manejo de paquetes. Asimismo, ROS también brinda herramientas y librerías que permiten construir, escribir y correr código a través y mediante varias computadoras.

ROS tiene una arquitectura modular que permite a los desarrolladores construir sistemas complejos robóticos combinando diferentes componentes. Por ende, la arquitectura de ROS 2 se basa en que los procesos, llamados nodos que intercambian datos e información mediante mensajes y a través de comunicación tipo publicación-suscripción. Este tipo de intercambio de información se realiza a través de tópicos o interacciones de solicitud-respuesta llamados servicios (Milvus, s.f.).

Entre los elementos principales que integran la red de comunicación de ROS2 se encuentran los nodos, que son programas que ejecutan tareas específicas como el sensado, control, procesamiento, entre otros. Otro elemento esencial son los tópicos que son canales por los que fluye la información, mientras que los mensajes representan las estructuras de datos utilizados para el intercambio de información entre nodos. Dentro de este esquema, los

publicadores envían datos hacia un tópico y los suscriptores reciben la información y la procesan, permitiendo comunicación entre diferentes elementos del sistema. Finalmente, los launch files son scripts que permiten ejecutar múltiples nodos y herramientas de visualización con una sola instrucción, lo que facilita la organización y ejecución de un sistema (Open Robotics, 2021).

## **Solución del problema (Metodología)**

Para cumplir los objetivos del mini reto se implementó un paquete de ROS2 denominado *signal\_processing*, el cual contiene dos nodos principales: un generador de señal senoidal y un procesador de señal, además de un archivo de lanzamiento ubicado en la carpeta *launch*, que automatiza la ejecución de ambos nodos junto con la herramienta de visualización *rqt\_plot*.

### **Metodología general:**

1. Se definió la estructura del paquete en ROS2, incluyendo el archivo *setup.py*, las entradas de consola para los nodos y la carpeta de launch para centralizar la ejecución del sistema.
2. Se diseñó un nodo publicador (*signal\_generator*) encargado de generar una señal senoidal en tiempo real y publicar tanto la señal como la variable de tiempo en tópicos separados.
3. Se desarrolló un nodo suscriptor (*process*) que recibe la información de tiempo y aplica un corrimiento de fase, cambio de amplitud y offset a la señal, publicando el resultado en un nuevo tópico.
4. Se creó un archivo de lanzamiento (*signal\_processing.launch.py*) para iniciar automáticamente los nodos de generación y procesamiento de señal, así como *rqt\_plot* con los parámetros de configuración correctos, permitiendo observar simultáneamente las señales original y procesada.

### **Descripción de los nodos y funciones**

El nodo *signal\_generator* se implementó como una clase que hereda de *rospy.node.Node* y se inicializa con el nombre de nodo *signal\_generator*. Dentro de su constructor se crean dos publicadores de tipo *Float32*, uno para el tópico */signal* (señal senoidal) y otro para el tópico */time* (tiempo asociado a la señal). Para generar la señal en tiempo real se almacena el tiempo de inicio con *self.get\_clock().now()* y se configura un temporizador a 10 Hz

(*create\_timer(0.1, self.timer\_callback)*), que ejecuta periódicamente la función *timer\_callback*. En esta función se calcula el tiempo transcurrido en segundos y se obtiene la señal como  $\sin(t)$  usando la librería *math*, se empaquetan los valores en mensajes de tipo *Float32* y se publican en los tópicos correspondientes, registrando en el log el par tiempo–señal en cada iteración.

El nodo *process* se implementó en la clase *SignalProcess*, también derivada de *Node*. Este nodo crea suscripciones a los tópicos */signal* y */time*, recibiendo mensajes *Float32* mediante las funciones de callback *signal\_callback* y *time\_callback*, donde se almacenan los valores en variables internas *self.signal* y *self.time*. Adicionalmente, se define un parámetro de fase fija *self.phase = math.pi / 4.0* y un publicador al tópico */proc\_signal* que enviará la señal procesada. Mediante un temporizador de 0.1 s se ejecuta periódicamente *timer\_callback*, que primero verifica que exista un valor de tiempo válido y luego calcula una nueva señal *shifted = sin(time + phase)*, aplicando después un cambio de amplitud y offset con la expresión *result = 0.5 \* (shifted + 1.0)*, para finalmente publicar este resultado en */proc\_signal* como un mensaje *Float32* y registrar el valor en consola.

### **Archivo de lanzamiento y seguimiento del código**

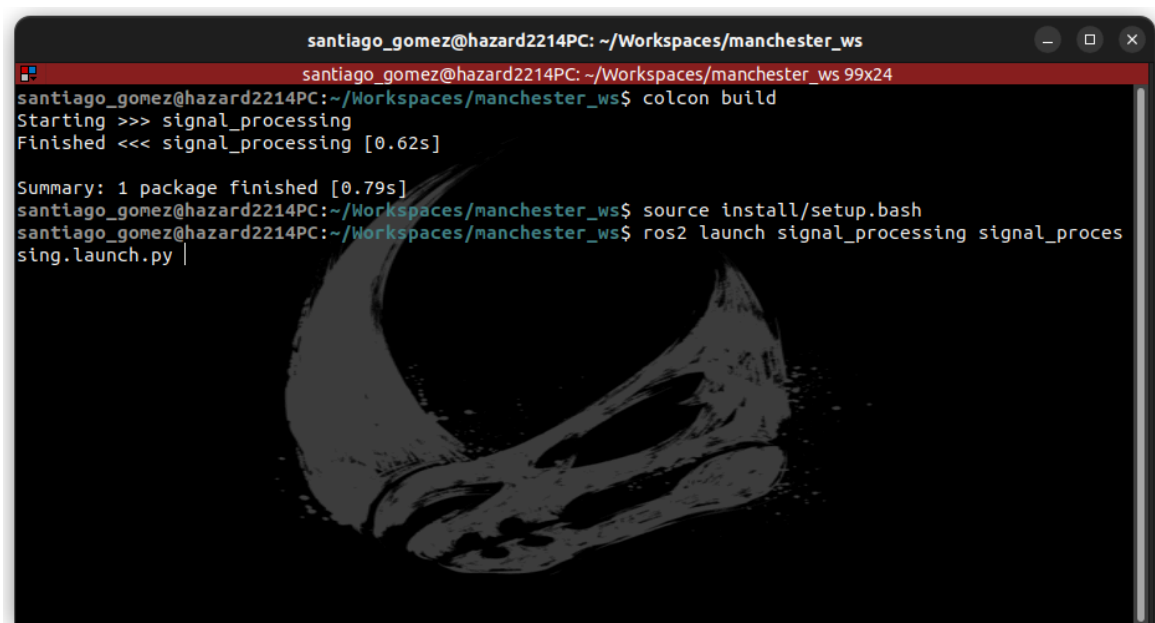
Para facilitar la ejecución integrada del sistema se desarrolló el archivo *signal\_processing.launch.py*. En este archivo se definen tres acciones de tipo *Node*: el nodo del paquete *signal\_procesing* con el ejecutable *signal\_generator*, el nodo del mismo paquete con el ejecutable *process* y un nodo de la herramienta *rqt\_plot* configurado con parámetros para visualizar los tópicos */signal/data* y */proc\_signal/data*. Finalmente, se construye un *LaunchDescription* que lanza de inmediato los nodos *signal\_generator* y *process*, y mediante un *TimerAction* se inicia *rqt\_plot* con un retardo de 2 segundos para asegurar que los tópicos ya están siendo publicados cuando se abra la interfaz de graficación.

La integración de los nodos en el sistema se completa con el archivo *setup.py*, donde se declara el nombre del paquete, los archivos de lanzamiento a instalar y, de forma importante, las entradas de consola *signal\_generator = signal\_procesing.signal\_generator:main* y *process = signal\_procesing.process:main*, que permiten que los ejecutables puedan ser invocados desde el archivo de launch y desde la línea de comandos. Con esta metodología se logró un flujo completo: generación de una señal senoidal en tiempo real, procesamiento de la misma con un corrimiento de fase y

modificación de amplitud/offset, y visualización simultánea de ambas señales para evaluar el comportamiento del sistema.

## Resultados

Primero que nada, se compiló el paquete realizado (*signal\_processing*) en la carpeta del *workspace* en que se está trabajando, usando el comando *colcon build*. Posteriormente, se configura el *workspace* para encontrar los paquetes y archivos necesarios para encontrar los archivos necesarios para el funcionamiento del paquete mediante el comando *source install/setup.bash*.

A terminal window titled 'santiago\_gomez@hazard2214PC: ~/Workspaces/manchester\_ws' with a red header bar. The terminal shows the execution of 'colcon build' for the 'signal\_processing' package, which finishes in 0.62s. A summary indicates 1 package finished in 0.79s. Subsequently, 'source install/setup.bash' is run, followed by 'ros2 launch signal\_processing signal\_processing.launch.py'. A faint skull watermark is visible in the background of the terminal.

```
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws 99x24
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ colcon build
Starting >>> signal_processing
Finished <<< signal_processing [0.62s]

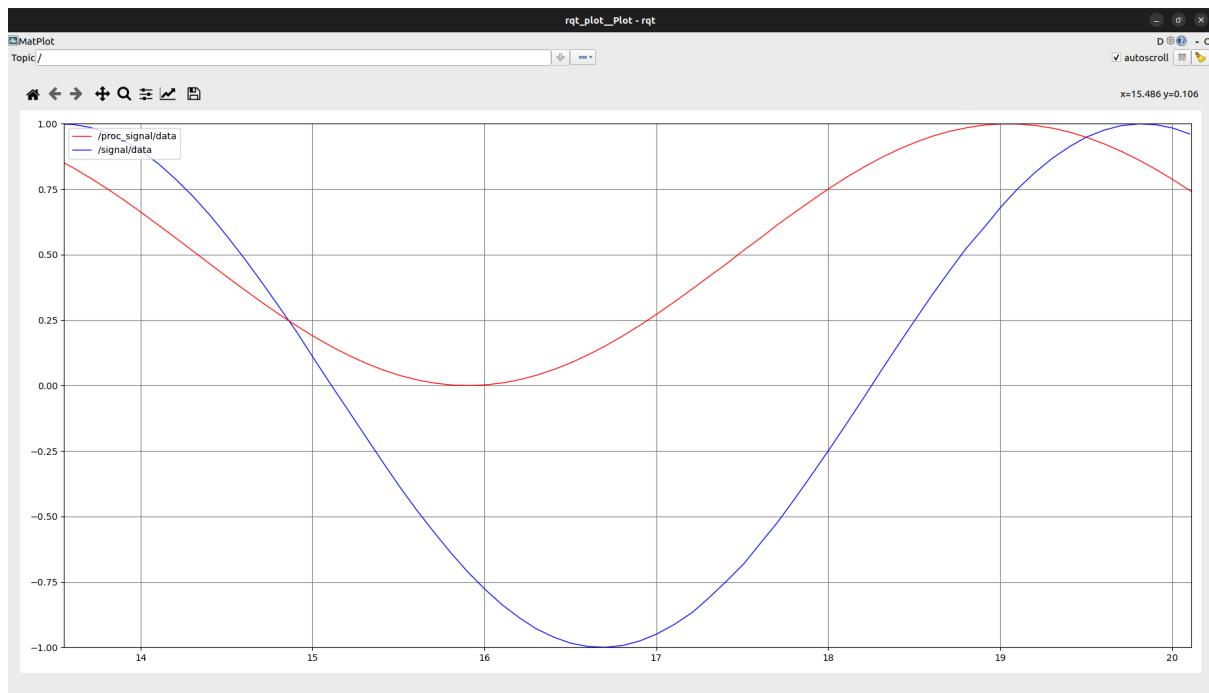
Summary: 1 package finished [0.79s]
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ source install/setup.bash
santiago_gomez@hazard2214PC:~/Workspaces/manchester_ws$ ros2 launch signal_processing signal_processing.launch.py |
```

**Imagen 1.** Compilación del paquete y configuración del espacio del trabajo.

Tras ejecutar el archivo de lanzamiento mediante *ros2 launch signal\_processing signal\_processing.launch.py*, se puede observar en la terminal cómo se imprimen la señal original y la señal procesada con respecto al tiempo, en forma de *log*. Asimismo, se abre una ventana de la *GUI* del paquete *rqt*, utilizando el plugin de *Plot* para representar gráficamente el comportamiento de ambas señales, utilizando la información *data* de los tópicos */signal* y */proc\_signal*.

```
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws
santiago_gomez@hazard2214PC: ~/Workspaces/manchester_ws 99x24
[process-2] [INFO] [1771450096.086072661] [process]: t=343.60 proc_signal=0.036
[signal_generator-1] [INFO] [1771450096.157214504] [signal_generator]: t=343.70 signal=-0.954
[process-2] [INFO] [1771450096.185746440] [process]: t=343.70 proc_signal=0.057
[signal_generator-1] [INFO] [1771450096.258886633] [signal_generator]: t=343.80 signal=-0.979
[process-2] [INFO] [1771450096.286054806] [process]: t=343.80 proc_signal=0.082
[signal_generator-1] [INFO] [1771450096.357160938] [signal_generator]: t=343.90 signal=-0.995
[process-2] [INFO] [1771450096.385989554] [process]: t=343.90 proc_signal=0.112
[signal_generator-1] [INFO] [1771450096.457001544] [signal_generator]: t=344.00 signal=-1.000
[process-2] [INFO] [1771450096.485517578] [process]: t=344.00 proc_signal=0.145
[signal_generator-1] [INFO] [1771450096.557116592] [signal_generator]: t=344.10 signal=-0.995
[process-2] [INFO] [1771450096.585593095] [process]: t=344.10 proc_signal=0.182
[signal_generator-1] [INFO] [1771450096.657436183] [signal_generator]: t=344.20 signal=-0.981
[process-2] [INFO] [1771450096.686273101] [process]: t=344.20 proc_signal=0.222
[signal_generator-1] [INFO] [1771450096.757243682] [signal_generator]: t=344.30 signal=-0.957
[process-2] [INFO] [1771450096.785627179] [process]: t=344.30 proc_signal=0.265
[signal_generator-1] [INFO] [1771450096.858272831] [signal_generator]: t=344.40 signal=-0.923
[process-2] [INFO] [1771450096.886391038] [process]: t=344.40 proc_signal=0.310
[signal_generator-1] [INFO] [1771450096.957033223] [signal_generator]: t=344.50 signal=-0.880
[process-2] [INFO] [1771450096.986082499] [process]: t=344.50 proc_signal=0.357
[signal_generator-1] [INFO] [1771450097.058604668] [signal_generator]: t=344.60 signal=-0.828
[process-2] [INFO] [1771450097.085939236] [process]: t=344.60 proc_signal=0.406
[signal_generator-1] [INFO] [1771450097.157280110] [signal_generator]: t=344.70 signal=-0.767
[process-2] [INFO] [1771450097.185755575] [process]: t=344.70 proc_signal=0.455
```

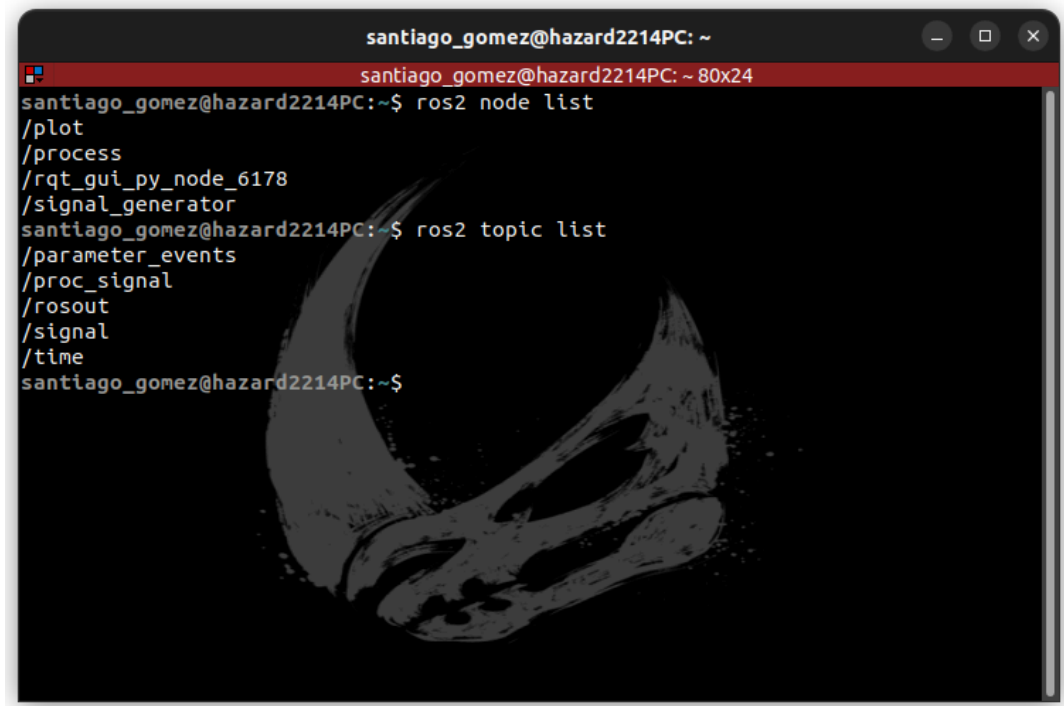
**Imagen 2.** Impresión en terminal de la información de los nodos de /signal\_generator y /process.



**Imagen 3.** Representación gráfica de las señales generadas.

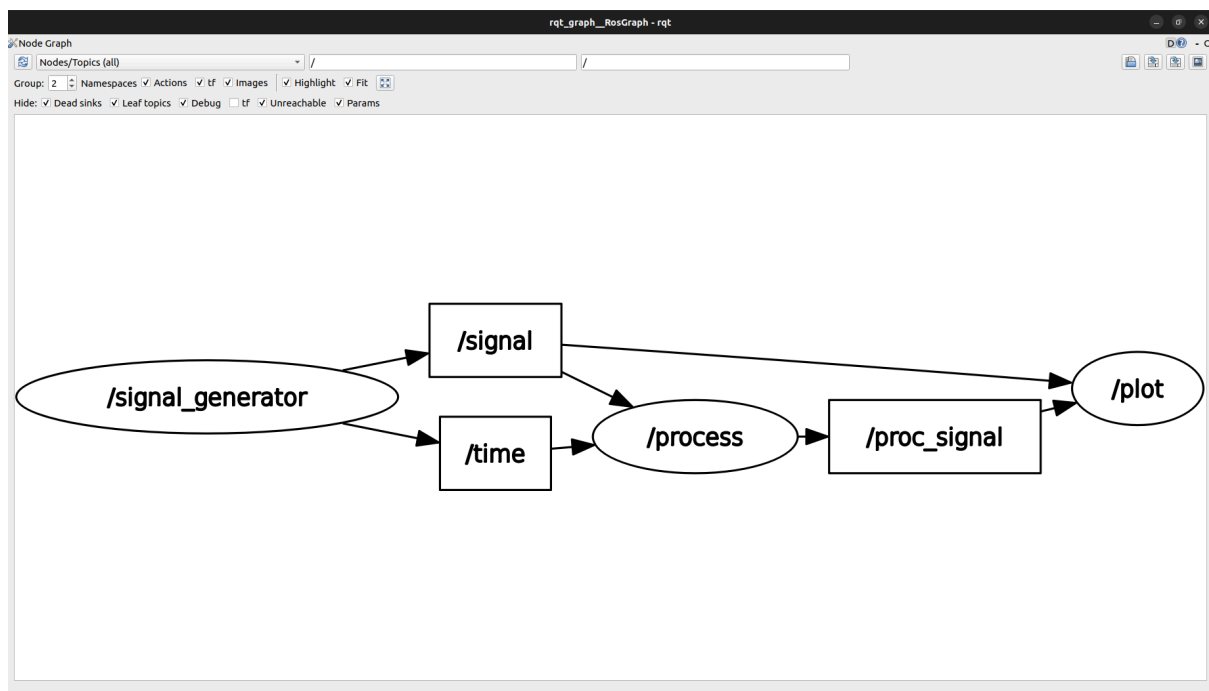
La línea azul representa la señal original, mientras que la línea roja representa la señal procesada. Se verificó que la señal procesada presenta un corrimiento de fase de  $\pi/4$  rad respecto a la señal original, así como una reducción de amplitud del 50% y un

desplazamiento vertical positivo de 0.5 unidades, lo cual coincide con la expresión implementada en el nodo *process*. Su funcionamiento se puede observar al verificar los nodos y tópicos presentes, al igual que cómo están interconectados.



```
santiago_gomez@hazard2214PC: ~  
santiago_gomez@hazard2214PC: ~ 80x24  
santiago_gomez@hazard2214PC:~$ ros2 node list  
/plot  
/process  
/rqt_gui_py_node_6178  
/signal_generator  
santiago_gomez@hazard2214PC:~$ ros2 topic list  
/parameter_events  
/proc_signal  
/rosout  
/signal  
/time  
santiago_gomez@hazard2214PC:~$
```

**Imagen 4.** Lista de nodos y tópicos en ejecución.



**Imagen 5.** Representación gráfica de cómo se están conectando los nodos y tópicos utilizando *rqt\_graph*.

El diagrama de *rqt\_graph* confirma que el nodo *signal\_generator* publica en los tópicos */signal* y */time*, mientras que el nodo *process* se suscribe a ambos y publica el resultado en */proc\_signal*, validando la arquitectura publicador–suscriptor implementada.

## Conclusiones

En conclusión, se logró la implementación de un sistema de generación, procesamiento y visualización de una señal senoidal utilizando ROS 2, cumpliendo con la arquitectura de comunicación de un *talker-listener*. En cuanto al cumplimiento de los objetivos, estos se alcanzaron de manera satisfactoria. Se logró que el nodo *signal\_generator* publicara la señal senoidal y el tiempo transcurrido en los tópicos correspondientes, mientras que el nodo *process* se suscribió correctamente a dicha información, aplicó el corrimiento de fase ( $\pi/4$  rad), la reducción de amplitud del 50 % y el desplazamiento vertical definido, y publicó la señal resultante en el tópico */proc\_signal*. La visualización simultánea en *rqt\_plot* permitió verificar gráficamente las transformaciones aplicadas, confirmando que los resultados obtenidos coinciden con la expresión matemática implementada. Asimismo, el uso de *rqt\_graph* validó la correcta configuración del modelo *talker-listener*.

La frecuencia de publicación utilizada (10 Hz) es suficiente para la demostración funcional de este ejemplo, pero puede generar una representación gráfica menos suave en comparación con frecuencias mayores. Esta implementación en concreto podría mejorarse al implementar una parametrización dinámica en los valores de fase, amplitud y offset, permitiendo modificar estos valores en tiempo de ejecución en tiempo real.

El desarrollo de este reto permitió consolidar la comprensión de la arquitectura de ROS 2, particularmente en el uso de nodos, tópicos, mensajes y archivos de lanzamiento. Además de cumplir con los objetivos planteados, la actividad ayudó a reforzar la relación entre el modelado matemático de señales y su implementación práctica en un entorno robótico distribuido.

## Referencias

Milvus. (s.f.). What is a ROS (Robot Operating System), and how is it used in robotics?

<https://milvus.io/ai-quick-reference/what-is-a-ros-robot-operating-system-and-how-is-it-used-in-robotics>

Open Robotics (2022). ROS Introduction. <https://wiki.ros.org/es/ROS/Introduccion>

Robotnik. (s.f.). ROS 2 (Robot Operating System): visión general y puntos clave del software de robótica.

<https://robotnik.eu/es/ros-2-robot-operating-system-vision-general-y-puntos-clave-del-software-de-robotica/>

Ultralytics. (s.f.). Guía de inicio rápido de ROS (Sistema Operativo Robótico).

<https://docs.ultralytics.com/es/guides/ros-quickstart/>